

```
> claude-code --masterprompts
```

300 Masterprompts für Claude Code

Eine kuratierte Referenz bewährter Prompt-Vorlagen für jede Phase der Softwareentwicklung — von der Architektur bis zum Pitch Deck, durchsucht wie eine Kommandozeile.

300

15

20

PROMPTS

KATEGORIEN

PRO KATEGORIE

\$ Einführung

Masterprompts sind sorgfältig formulierte, wiederverwendbare Anfragen, die als zuverlässiger Ausgangspunkt für die Arbeit mit Claude Code dienen. Statt jedes Mal neu zu überlegen, wie man eine Aufgabe formuliert, greifst du auf eine bewährte Struktur zurück und passt sie an dein konkretes Projekt an.

Wie nutzt man sie effektiv?

- Jeder Prompt ist mit einem konkreten Beispiel ausformuliert — der orange unterstrichene Teil markiert, was du durch dein eigenes Projekt, deine Technologie oder deinen Code ersetzt.
- Kombiniere mehrere Prompts nacheinander, um komplexe Aufgaben Schritt für Schritt zu lösen.
- Gib zusätzlichen Kontext mit: Framework, Programmiersprache, Codebasis-Ausschnitte oder bestehende Konventionen.

Tipps für optimale Ergebnisse

- Sei so spezifisch wie möglich — konkrete Anforderungen liefern konkrete Ergebnisse.
- Nutze Folgefragen, um Ergebnisse zu verfeinern statt von vorn zu beginnen.
- Lasse dir bei komplexen Aufgaben zuerst einen Plan erstellen, bevor Code generiert wird.

Beispielprojekt: Online-Marktplatz für handgefertigte Möbel

01.01

> Erstelle eine Projektstruktur für einen Online-Marktplatz für handgefertigte Möbel mit Next.js und einem separaten Node.js-Backend.

01.02

> Entwirf eine Microservices-Architektur für den Marktplatz mit getrennten Services für Produkte, Bestellungen, Zahlungen und Benachrichtigungen.

01.03

> Erstelle ein ER-Diagramm für Verkäufer, Produkte, Bestellungen und Bewertungen im Möbelmarktplatz.

01.04

> Definiere den Tech-Stack für eine skalierbare E-Commerce-Plattform mit erwarteten 50.000 Nutzern im ersten Jahr.

01.05

> Erstelle ein Domain-Modell für den Bestellprozess vom Warenkorb bis zur Auslieferung.

01.06

> Entwirf eine CI/CD-Pipeline für das Marktplatz-Backend mit automatisierten Tests und Staging-Deployment auf AWS.

01.07

> Erstelle eine API-Dokumentation für die Produkt- und Bestell-Endpoints des Marktplatzes.

01.08

> Definiere die Datenbank-Schemata für Nutzer, Produkte, Bestellungen und Zahlungen in PostgreSQL.

01.09

> Erstelle ein Security-Konzept für die Zahlungsabwicklung und Nutzerdaten im Marktplatz.

01.10

> Entwirf eine Monitoring-Strategie für Verfügbarkeit und Antwortzeiten der Marktplatz-APIs.

01.11

>Erstelle eine README-Vorlage für den Marktplatz-Client, der handgefertigte Möbel von unabhängigen Tischlereien präsentiert.

01.12

>Definiere Coding-Guidelines für die TypeScript-Codebasis des Bestell- und Zahlungsservices im Möbelmarktplatz.

01.13

>Erstelle ein Glossar der Fachbegriffe für den Handel mit handgefertigten Möbeln, etwa Werkstattprofil, Losgröße und Sonderanfertigung.

01.14

>Entwirf eine Ordnerstruktur für ein Monorepo mit Next.js-Storefront, Verkäufer-Dashboard und Node.js-Backend für den Möbelmarktplatz.

01.15

>Erstelle eine Risikoanalyse für den technischen Aufbau von der Zahlungs- und Versandabwicklung zwischen Käufern und einzelnen Möbeltischlereien.

01.16

>Definiere Nicht-funktionale Anforderungen für Ladezeiten und Bildqualität der Produktgalerien handgefertigter Möbelstücke.

01.17

>Erstelle ein Architektur-Entscheidungsprotokoll (ADR) für die Wahl zwischen monolithischem Checkout und separatem Bestellservice im Möbelmarktplatz.

01.18

>Entwirf eine Skalierungsstrategie für saisonale Nachfragespitzen im Weihnachtsgeschäft des Möbelmarktplatzes.

01.19

>Erstelle einen Onboarding-Guide für neue Entwickler bei dem Team, das das Verkäufer- Dashboard für Möbeltischlereien pflegt.

01.20

>Definiere eine Versionierungsstrategie für die API von dem Bestell- und Lagerbestandsservice des Möbelmarktplatzes.

02.01

>Generiere eine Klasse für eine Aufgabe (Task) mit Titel, Fälligkeitsdatum, Priorität und Status.

02.02

>Refactoriere diesen Code: `function getTasks(u){var r=[];for(var i=0;i<tasks.length;i++){if(tasks[i].user==u){r.push(tasks[i])}}return r}`

02.03

>Erstelle ein Interface für ein Repository, das Aufgaben speichert, lädt und aktualisiert.

02.04

>Konvertiere diesen Code von JavaScript nach TypeScript: `function addTask(title, dueDate) { return { title, dueDate, done: false }; }`

02.05

>Erstelle eine Utility-Funktion für die Formatierung relativer Fälligkeitsdaten (z. B. „in 2 Tagen“, „überfällig“).

02.06

>Optimiere diesen Code für Performance: `tasks.filter(t => !t.done).sort((a,b) => new Date(a.due) - new Date(b.due)).map(t => renderTask(t))`

02.07

>Erstelle einen Decorator für das Protokollieren von Ausführungszeit und Fehlern im Task-Service.

02.08

>Refactoriere diese Funktion für Lesbarkeit: `function f(t){return t.p==1&&!t.d&&new Date(t.due)<new Date()}}`

02.09

>Erstelle einen Mixin für Sortier- und Filterlogik, die von mehreren Listen-Komponenten wiederverwendet wird.

02.10

>Generiere einen Service für die Zwei-Wege-Synchronisation von Aufgaben mit dem Google-Kalender.

02.11

>Erstelle eine Factory-Funktion für das Erzeugen neuer Task-Objekte mit Standardwerten für Status, Priorität und Erstellungsdatum.

02.12

>Extrahiere eine wiederverwendbare Komponente aus diesem Code:

```
function renderTaskInList(t){ return '<li>' + t.title + ' (' + t.priority + ')</li>'; } function renderTaskInBoard(t){ return '<div>' + t.title + ' (' + t.priority + ')</div>'; }
```

02.13

>Erstelle einen Custom Hook für das Laden, Zwischenspeichern und automatische Aktualisieren der Aufgabenliste eines angemeldeten Nutzers.

02.14

>Vereinheitliche den Fehlerbehandlungs-Stil in diesem Code:

```
function createTask(data){ try { return api.post('/tasks', data); } catch(e){ console.log(e); } } function deleteTask(id){ return api.delete('/tasks/' + id).catch(function(err){ alert('Fehler: ' + err); }); }
```

02.15

>Erstelle ein Value Object für eine Fälligkeitsfrist, die Datum und Uhrzeit kapselt und Methoden zur Prüfung von Überfälligkeit bereitstellt.

02.16

>Entferne toten Code aus diesem Modul:

```
function sortTasksByDate(tasks){ return tasks.sort(function(a,b){ return a.dueDate - b.dueDate; }); } function sortTasksByDateOld(tasks){ return tasks; } function logDebug(x){ if(false){ console.log('debug', x); } }
```

02.17

>Erstelle einen Builder für komplexe Task-Filter, die Status, Priorität, Fälligkeitsdatum und zugewiesenen Bearbeiter kombinieren.

02.18

>Modularisiere diese Datei in kleinere Einheiten:

```
function addTask(t){
  tasks.push(t);
}
function removeTask(id){
  tasks = tasks.filter(function(t){
    return t.id !== id;
  });
}
function addUser(u){
  users.push(u);
}
function sendNotification(msg){
  notifications.push(msg);
}
function renderTaskList(){
  return tasks.map(function(t){
    return t.title;
  });
}
```

02.19

>Erstelle eine Validierungsfunktion für die Eingabedaten beim Anlegen einer neuen Aufgabe, einschließlich Pflichtfeldern und Datumsformat.

02.20

>Vereinfache diese verschachtelte Bedingung:

```
if(task.status === 'open'){
  if(task.priority === 'high'){
    if(task.dueDate < today){
      showAlert = true;
    }
  }
}
```

03.01

>Erstelle Unit-Tests für die Rabattberechnung im Checkout-Prozess (Prozent-, Fest- und Versandrabatte).

03.02

>Schreibe Integrationstests für den kompletten Bestellablauf vom Warenkorb bis zur Zahlungsbestätigung.

03.03

>Erstelle einen Debugging-Guide für doppelte Bestellungen bei gleichzeitigen Checkout-Anfragen.

03.04

>Generiere Testdaten für 200 Bestellungen mit unterschiedlichen Zahlungsarten und Versandländern.

03.05

>Erstelle einen Testplan für den Release der neuen Gutschein-Funktion im Checkout.

03.06

>Simuliere einen Fehlerfall für einen Zahlungsdienstleister, der während des Checkouts nicht antwortet.

03.07

>Erstelle einen Mock für den externen Zahlungsanbieter Stripe im Checkout-Test.

03.08

>Schreibe End-to-End-Tests für den Kaufabschluss als Gast ohne Benutzerkonto.

03.09

>Erstelle ein Performance-Test-Skript für 1000 gleichzeitige Checkout-Anfragen am Black-Friday-Peak.

03.10

>Erstelle einen Security-Test-Report für die Eingabefelder im Zahlungsformular (XSS, SQL-Injection, CSRF).

03.11

> Erstelle Snapshot-Tests für die Zusammenfassungskomponente der Bestellübersicht im letzten Checkout-Schritt (Artikel, Preise, Versandkosten, Gesamtsumme).

03.12

> Schreibe Regressionstests für einen behobenen Bug in der Rabattcode-Validierung, bei dem abgelaufene Gutscheine fälschlicherweise akzeptiert wurden.

03.13

> Erstelle eine Teststrategie für die Migration von der alten Stripe-Checkout-Sessions-Integration auf die neue Payment-Intents-API mit Strong Customer Authentication.

03.14

> Analysiere diesen Stacktrace und finde die Ursache: `TypeError: Cannot read properties of undefined (reading 'postalCode')`

```
    at calculateShippingCost
    (checkout/shipping.js:47:22)
    at
    CheckoutController.applyShippingRate
    (checkout/controller.js:118:9)
    at async
    CheckoutController.submitOrder
    (checkout/controller.js:76:5)
    at async processCheckoutRequest
    (checkout/routes.js:33:3) .
```

03.15

> Erstelle Contract-Tests zwischen dem Checkout-Service und dem Payment-Gateway-Service (Request-/Response-Schema für Zahlungsautorisierung und Rückerstattungen).

03.16

> Schreibe Tests für Edge Cases von der Kombination aus Gutschein- und Geschenkkartencodes im Warenkorb (mehrere Codes, Restguthaben, Überzahlung).

03.17

>Erstelle einen Chaos-Engineering-Testfall für den Ausfall des Zahlungsdienstleisters während der Zahlungsautorisierung (simulierte Timeouts und 500er-Antworten).

03.18

>Miss die Testabdeckung von dem Adressvalidierungsmodul im Checkout (Pflichtfelder, Postleitzahlenformate, Länderprüfung) und identifiziere Lücken.

03.19

>Erstelle einen Reproduktionsschritt-Report für einen gemeldeten Bug in der Gesamtsummenanzeige, bei dem nach einem Währungswechsel im Checkout falsche Beträge erscheinen.

03.20

>Schreibe visuelle Regressionstests für die Checkout-Zusammenfassungsseite auf Mobile-, Tablet- und Desktop-Breakpoints.

04.01

> Erstelle eine REST-API für eine Rezept-Plattform mit Suche, Bewertungen und Favoritenliste.

04.02

> Entwirf eine GraphQL-API für das Abfragen von Rezepten inklusive Zutaten, Nährwerten und Bewertungen in einer Anfrage.

04.03

> Erstelle eine API-Dokumentation für die öffentlichen Rezept- und Bewertungs-Endpoints.

04.04

> Definiere API-Endpoints für das Anlegen, Bearbeiten und Löschen eigener Rezepte.

04.05

> Erstelle einen API-Client für den Zugriff auf die Rezept-API aus einer mobilen App.

04.06

> Implementiere Authentifizierung für die Rezept-API mit JWT und Refresh-Tokens.

04.07

> Erstelle eine Rate-Limiting-Strategie für die öffentliche Such-API, um Scraping zu verhindern.

04.08

> Erstelle einen Webhook für Benachrichtigungen, wenn ein Rezept eine neue Bewertung erhält.

04.09

> Simuliere API-Antworten für die Rezept-Suche während der Frontend-Entwicklung, bevor das Backend fertig ist.

04.10

> Erstelle eine OpenAPI-Spezifikation für alle Endpoints der Rezept-Plattform.

04.11

> Erstelle eine Versionierungsstrategie für die Rezept-API mit Bewertungsfunktion beim Wechsel von v1 auf v2.

04.12

> Entwirf eine Pagination-Strategie für die Liste der Rezeptbewertungen mit Cursor-basierter Navigation.

04.13

> Erstelle einen GraphQL-Resolver für Rezepte mit verschachtelten Bewertungen und berechneter Durchschnittsnote.

04.14

> Implementiere Idempotenz für den Endpoint zum Absenden einer neuen Rezeptbewertung.

04.15

> Erstelle eine Fehlerantwort-Konvention für die Rezept-API bei ungültigen oder doppelten Bewertungen.

04.16

> Entwirf eine Bulk-Import-Schnittstelle für das Einspielen bestehender Rezepte samt historischer Bewertungsdaten.

04.17

> Erstelle einen SDK-Wrapper für die Rezept-API in TypeScript zum Abrufen von Rezepten und Bewertungen.

04.18

> Implementiere Caching-Header für die Endpunkte zum Abrufen von Rezeptdetails und Bewertungsübersichten.

04.19

> Erstelle eine Deprecation-Strategie für einen alten Endpoint in der Rezept-API, der Bewertungen im veralteten Fünf-Sterne-Format zurückgibt.

04.20

> Entwirf eine Multi-Tenancy-Strategie für die API von einer Rezept-Plattform, die mehrere Restaurantketten mit jeweils eigenen Bewertungssystemen bedient.

05.01

> Erstelle eine React-Komponente für eine Fortschrittsanzeige des täglichen Schritteziels.

05.02

> Entwirf ein CSS-Layout für den Trainingsplan-Kalender mit Wochen- und Monatsansicht.

05.03

> Erstelle eine Animation für das Erreichen eines neuen persönlichen Rekords beim Training.

05.04

> Erstelle eine responsive Navigation für die Fitness-App mit Tab-Bar auf Mobile und Sidebar auf Desktop.

05.05

> Entwirf ein Dashboard für die wöchentliche Trainingsübersicht mit Kalorien, Distanz und Herzfrequenz.

05.06

> Erstelle ein Formular für das Anlegen eines neuen Trainingsplans mit Übungen, Sätzen und Wiederholungen.

05.07

> Entwirf eine Landingpage für den Launch der Fitness-App mit Fokus auf die Community-Funktion.

05.08

> Erstelle ein Dark-Mode-Design für die Trainingsansicht, die häufig frühmorgens genutzt wird.

05.09

> Erstelle eine mobile Version für die Desktop-Statistikseite mit Trainingsverlauf.

05.10

> Entwirf eine interaktive Komponente für das Ziehen und Neuordnen von Übungen im Trainingsplan.

05.11

>Erstelle eine Skeleton-Loading-Komponente für die Ladeansicht des wöchentlichen Trainingsverlaufs mit Kalorien- und Schrittstatistiken.

05.12

>Entwirf einen Empty State für den Bereich, in dem noch kein einziges Training aufgezeichnet wurde.

05.13

>Erstelle eine Toast-Benachrichtigungs-Komponente für das Erreichen eines täglichen Schritt- oder Kalorienziels.

05.14

>Entwirf einen Onboarding-Flow für die erste Erfassung von Körperdaten, Fitnesszielen und bevorzugten Trainingsarten.

05.15

>Erstelle eine barrierefreie Modal-Komponente für die Bestätigung beim Löschen eines gespeicherten Trainingsplans.

05.16

>Entwirf ein Drag-and-Drop-Interface für das Neuordnen von Übungen innerhalb eines individuellen Trainingsplans.

05.17

>Erstelle eine Infinite-Scroll-Liste für den chronologischen Verlauf abgeschlossener Trainingseinheiten.

05.18

>Entwirf eine Fehlerseite für den Fall eines Verbindungsabbruchs zur Smartwatch während der Datensynchronisierung.

05.19

>Erstelle eine Filteroberfläche für das Durchsuchen der Übungsbibliothek nach Muskelgruppe, Gerät und Schwierigkeitsgrad.

05.20

>Entwirf eine mehrstufige Formular-Wizard-Komponente für die schrittweise Erstellung eines personalisierten Trainingsplans.

06.01

> Erstelle ein Datenbank-Modell für Unterkünfte, Verfügbarkeiten, Buchungen und Gastprofile.

06.02

> Implementiere eine Datenbank-Query für die Suche nach verfügbaren Unterkünften in einem Zeitraum und einer Region.

06.03

> Erstelle eine Datenbank-Migration für das Hinzufügen einer Stornierungsfrist zu bestehenden Buchungen.

06.04

> Entwirf eine Datenbank-Indexierung für schnelle Verfügbarkeitsabfragen bei über 100.000 Unterkünften.

06.05

> Erstelle eine Caching-Strategie für häufig abgerufene Unterkunftsdetails und Bildergalerien.

06.06

> Implementiere eine Datenbank-Transaktion für das gleichzeitige Anlegen einer Buchung und das Blockieren des Zeitraums.

06.07

> Erstelle einen Datenbank-Seeder für 100 Beispiel-Unterkünfte mit Buchungen für die lokale Entwicklung.

06.08

> Entwirf eine Datenbank-Backup-Strategie für die Buchungs- und Zahlungsdaten mit täglichen Snapshots.

06.09

> Erstelle eine Datenbank-Optimierung für die langsame Abfrage der Buchungshistorie eines Gastes.

06.10

> Implementiere eine Datenbank-Replikation für die Lesezugriffe der Suchfunktion, getrennt vom Schreibbetrieb der Buchungen.

06.11

>Entwirf eine Soft-Delete-Strategie für stornierte Buchungen und deaktivierte Unterkunftsinserate mit 30-tägiger Wiederherstellungsfrist.

06.12

>Erstelle eine Multi-Tenancy-Datenbankstruktur für mehrere Vermieter mit jeweils eigenen Unterkünften, Preislisten und Buchungskalendern.

06.13

>Implementiere Full-Text-Suche für Unterkunftsbeschreibungen, Ausstattungsmerkmale und Standortangaben.

06.14

>Entwirf eine Archivierungsstrategie für alte Daten in abgeschlossenen Buchungen und Gastbewertungen, die älter als zwei Jahre sind.

06.15

>Erstelle eine Datenvalidierungsschicht für Buchungsanfragen, Belegungszeiträume und die maximale Gästeanzahl pro Ferienwohnung.

06.16

>Implementiere Optimistic Locking für gleichzeitige Buchungsversuche auf denselben Verfügbarkeitszeitraum einer Ferienwohnung.

06.17

>Entwirf eine Sharding-Strategie für Buchungsdaten nach geografischen Regionen der Ferienwohnungen.

06.18

>Erstelle eine Datenanonymisierung für DSGVO-Anfragen in Gastprofilen, Zahlungsdaten und Nachrichtenverläufen mit Vermietern.

06.19

>Implementiere eine Job-Queue für asynchrone Verarbeitung in Buchungsbestätigungen, Zahlungsabgleich und Versand von Check-in-Informationen.

06.20

>Entwirf ein Audit-Trail-Schema für Änderungen an Preisen, Verfügbarkeiten und Stornierungsbedingungen von Unterkunftsinseraten.

07.01

>Erstelle ein Dockerfile für einen Node.js-Microservice mit mehrstufigem Build für kleinere Images.

07.02

>Entwirf eine Kubernetes-Konfiguration für drei Microservices mit eigenen Deployments, Services und Health-Checks.

07.03

>Erstelle ein Deployment-Script für das Rolling-Update der Microservices ohne Downtime.

07.04

>Definiere eine CI/CD-Pipeline für automatische Tests, Docker-Build und Deployment bei jedem Merge in main.

07.05

>Erstelle eine Monitoring-Konfiguration für CPU-, Speicher- und Latenz-Metriken aller Microservices mit Prometheus.

07.06

>Entwirf eine Logging-Strategie für zentrales, durchsuchbares Logging über alle Microservices hinweg.

07.07

>Erstelle ein Backup-Script für die Konfigurationsdaten und Secrets des Kubernetes-Clusters.

07.08

>Definiere eine Disaster-Recovery-Strategie für den Ausfall einer kompletten Availability-Zone.

07.09

>Erstelle ein Skript für den Server-Setup eines neuen Staging-Servers mit Docker, Nginx und SSL-Zertifikaten.

07.10

>Entwirf eine Load-Balancing-Strategie für den Traffic zwischen mehreren Instanzen des Bestell-Services.

07.11

>Erstelle eine Blue-Green-Deployment-Strategie für den Zahlungsabwicklungs-Service in der Kubernetes-Umgebung.

07.12

>Entwirf eine Secrets-Management-Lösung für die Datenbank-Zugangsdaten und API-Schlüssel aller Microservices mit HashiCorp Vault.

07.13

>Erstelle ein Infrastructure-as-Code-Setup mit Terraform für die AWS-ECS-Cluster der Bestell- und Lager-Services.

07.14

>Definiere Rollback-Kriterien für Deployments von dem Benutzerauthentifizierungs-Service bei erhöhten Fehlerraten nach einem Release.

07.15

>Erstelle eine Kostenoptimierungs-Analyse für die Infrastruktur von den containerisierten Microservices auf AWS Fargate.

07.16

>Entwirf eine Multi-Environment-Strategie (Dev/Staging/Prod) für den API-Gateway und die dahinterliegenden Microservices.

07.17

>Erstelle ein Health-Check-Endpoint-Konzept für den Bestell-Service mit Liveness- und Readiness-Probes.

07.18

>Definiere Alerting-Regeln für kritische Metriken von der Nachrichtenwarteschlange zwischen Inventar- und Versand-Service.

07.19

>Erstelle eine Autoscaling-Konfiguration für den Produktkatalog-Service bei saisonalen Traffic-Spitzen.

07.20

>Entwirf eine Zero-Downtime-Migrationsstrategie für die Umstellung der Bestelldatenbank von MongoDB auf PostgreSQL.

08.01

> Erstelle ein Security-Audit für die Login- und Überweisungsfunktion der Banking-App.

08.02

> Implementiere eine Authentifizierung für die Banking-App mit Zwei-Faktor-Authentifizierung per App und SMS.

08.03

> Erstelle ein Permissions-System für unterschiedliche Rollen wie Kontoinhaber, Prokurist und Kundenservice.

08.04

> Definiere eine Verschlüsselungs-Strategie für gespeicherte Kontodaten und Transaktionshistorien.

08.05

> Erstelle einen Performance-Testing-Report für die Kontoübersicht bei 10.000 gleichzeitigen Nutzern.

08.06

> Optimierte die Ladezeit für die Transaktionshistorie mit mehreren tausend Einträgen pro Konto.

08.07

> Erstelle einen Security-Report für verdächtige Login-Versuche und ungewöhnliche Überweisungsmuster.

08.08

> Implementiere eine Input-Validierung für das Überweisungsformular (IBAN, Betrag, Verwendungszweck).

08.09

> Erstelle ein Audit-Log für alle Kontoänderungen und Überweisungen zur Nachvollziehbarkeit.

08.10

> Definiere eine Threat-Modellierung für die mobile Banking-App inklusive Geräteverlust und Phishing.

08.11

>Erstelle eine OWASP-Top-10-Checkliste für die Authentifizierungs- und Zahlungsverkehrsmodule der Banking-App.

08.12

>Implementiere Content-Security-Policy-Header für das Online-Banking-Frontend zur Abwehr von Cross-Site-Scripting.

08.13

>Entwirf eine Session-Management-Strategie für eingeloggte Nutzer im Kundenportal der Banking-App inklusive automatischem Timeout.

08.14

>Erstelle einen Penetrationstest-Plan für die REST-API der Banking-App, die Kontostände und Überweisungen verarbeitet.

08.15

>Implementiere Rate-Limiting gegen Brute-Force-Angriffe auf den Login-Endpunkt und die PIN-Eingabe der Banking-App.

08.16

>Erstelle eine Dependency-Audit-Strategie für die Backend-Bibliotheken der Banking-App, die Zahlungsdaten verarbeiten.

08.17

>Optimiere die Bundle-Größe von der mobilen Web-Anwendung der Banking-App für schnellere Ladezeiten im Kontoüberblick.

08.18

>Erstelle eine Incident-Response-Checkliste für einen Sicherheitsvorfall bei einem Datenleck von Kundendaten der Banking-App.

08.19

>Implementiere sichere Passwort-Richtlinien für die Kundenkonten der Banking-App inklusive Zwei-Faktor-Authentifizierung.

08.20

>Erstelle eine Lasttest-Strategie für die Überweisungs- und Kontoauszugsfunktionen der Banking-App bei Spitzenlast am Monatsende.

09.01

>Erstelle ein Design-System für eine Meditations-App mit ruhigen Farben, Typografie und wiederverwendbaren Komponenten.

09.02

>Entwirf eine User Journey für einen neuen Nutzer von der Installation bis zur ersten abgeschlossenen Meditation.

09.03

>Erstelle eine Persona für eine gestresste Berufstätige, die abends zur Ruhe kommen möchte.

09.04

>Definiere einen Styleguide für Ton und Sprache der App-Texte (beruhigend, ermutigend, nie belehrend).

09.05

>Erstelle eine Accessibility-Checkliste für die audio-geführten Meditationen inklusive Untertiteln und Screenreader-Support.

09.06

>Entwirf ein User-Flow-Diagramm für das Erstellen einer personalisierten Meditationsroutine.

09.07

>Erstelle ein Moodboard für die visuelle Sprache der App: warm, minimalistisch, naturverbunden.

09.08

>Definiere einen Design-Sprint für die Entwicklung einer neuen Schlaf-Geschichten-Funktion in fünf Tagen.

09.09

>Erstelle einen Prototyp für den Onboarding-Flow mit Auswahl der Meditationsziele.

09.10

>Entwirf eine User-Testing-Strategie für die neue Atemübungs-Funktion vor dem Rollout.

09.11

>Erstelle ein Komponenten-Naming-Schema für die UI-Bausteine einer Meditations-App, etwa Session-Karten, Atem-Timer und Fortschrittsringe.

09.12

>Entwirf ein Fehlerzustands-Konzept für fehlgeschlagene Audio-Downloads und Verbindungsabbrüche während einer geführten Meditation.

09.13

>Erstelle eine Micro-Copy-Sammlung für Begrüßungstexte, Erinnerungen und Abschlussmeldungen einer Meditations-App.

09.14

>Definiere Interaktionsmuster für Gesten in dem Session-Player einer Meditations-App, etwa Wischen zum Überspringen und Halten zum Pausieren.

09.15

>Erstelle eine Informationsarchitektur für die Kursbibliothek einer Meditations-App mit Kategorien wie Schlaf, Stressabbau und Atemübungen.

09.16

>Entwirf ein Feedback-Konzept nach Nutzeraktionen in einer Meditations-App, etwa nach Abschluss einer Session oder dem Erreichen einer Streak.

09.17

>Erstelle Wireframes für den Onboarding-Flow einer Meditations-App vom ersten Start bis zur ersten abgeschlossenen Session.

09.18

>Definiere ein Icon-System für die Navigationsleiste und Funktionsbereiche einer Meditations-App, etwa Startseite, Bibliothek, Statistiken und Profil.

09.19

>Erstelle eine Usability-Test-Checkliste für den Buchungs- und Startprozess einer geführten Meditationssession.

09.20

>Entwirf ein Empty-State-Illustrationskonzept für eine leere Favoritenliste oder eine noch nicht begonnene Meditationsreise in der App.

10.01

>Erstelle einen Business-Plan für ein B2B-SaaS-Tool für Projektmanagement in kleinen Agenturen.

10.02

>Definiere eine Marketing-Strategie für die Positionierung gegenüber etablierten Tools wie Asana und Trello.

10.03

>Erstelle eine Roadmap für die ersten zwölf Monate nach dem Launch mit Kernfunktionen und Erweiterungen.

10.04

>Definiere eine Pricing-Strategie für drei Abostufen je nach Teamgröße und Funktionsumfang.

10.05

>Erstelle eine SWOT-Analyse für den Markteintritt in einem von wenigen großen Anbietern dominierten Markt.

10.06

>Entwirf eine Go-to-Market-Strategie für den Launch bei kleinen Design- und Marketingagenturen.

10.07

>Erstelle einen Kunden-Entwicklungsplan für die Validierung des Funktionsumfangs mit zehn Pilotkunden.

10.08

>Definiere eine Vertriebsstrategie für den Self-Service-Verkauf ohne eigenes Sales-Team.

10.09

>Erstelle ein Pitch-Deck für eine Seed-Finanzierungsrunde von 500.000 Euro.

10.10

>Definiere eine Produkt-Roadmap für die Einführung von Zeiterfassung und Rechnungsstellung als Erweiterung.

10.11

>Erstelle eine Kundenreferenz-Fallstudie für eine mittelständische Werbeagentur, die ihre Projektlaufzeiten mit dem Tool um 30 Prozent verkürzt hat.

10.12

>Definiere Erfolgskennzahlen (KPIs) für die Kundenbindung und Produktnutzung im ersten Jahr nach der Einführung des Tools.

10.13

>Erstelle eine Wettbewerbsanalyse für die drei führenden Anbieter von B2B-Projektmanagement-Software im deutschsprachigen Markt.

10.14

>Definiere eine Churn-Reduktionsstrategie für Kunden im mittleren Preissegment, die nach Ablauf der kostenlosen Testphase abspringen.

10.15

>Erstelle einen Onboarding-E-Mail-Funnel für neu registrierte Teams in den ersten 14 Tagen nach dem Sign-up.

10.16

>Definiere eine Partnerschaftsstrategie für Integrationen mit gängigen CRM- und Buchhaltungssystemen wie HubSpot und DATEV.

10.17

>Erstelle eine Investoren-Update-Vorlage für monatliche Fortschrittsberichte an die Seed-Investoren nach der Finanzierungsrunde.

10.18

>Definiere eine Feature-Priorisierungsmatrix für die nächste Produkt-Roadmap auf Basis von Kundenfeedback und Entwicklungsaufwand.

10.19

>Erstelle eine Kundenzufriedenheits-Umfrage für Power-User nach den ersten drei Monaten aktiver Nutzung.

10.20

>Definiere eine Internationalisierungsstrategie für den Markteintritt von unserem Projektmanagement-Tool für kleine und mittlere Agenturen in neue Länder.

11.01

> Erstelle eine Projektstruktur für eine Mitfahrgelegenheiten-App mit getrennten Modulen für Fahrer, Mitfahrer und Routenabgleich.

11.02

> Entwirf einen Onboarding-Flow für neue Nutzer, die Arbeitsweg, Wunschzeiten und Fahrzeugdaten hinterlegen.

11.03

> Implementiere Push-Benachrichtigungen für Fahrtanfragen, Abholzeit-Erinnerungen und Routenänderungen.

11.04

> Erstelle eine Offline-Synchronisation für gebuchte Fahrten und Chatnachrichten bei fehlender Netzverbindung während der Fahrt.

11.05

> Optimierte die App-Startzeit von der Kartenansicht mit den täglich verfügbaren Mitfahrgelegenheiten.

11.06

> Implementiere biometrische Authentifizierung für den Login vor Fahrtantritt und die Bestätigung von Zahlungen.

11.07

> Erstelle eine Deep-Linking-Strategie für geteilte Fahrtangebote, die Empfänger direkt zur Buchungsseite führen.

11.08

> Entwirf eine plattformübergreifende Komponente (iOS/Android) für die Anzeige des Live-Standorts von Fahrer und Mitfahrer auf der Karte.

11.09

> Implementiere In-App-Käufe für Premium-Mitgliedschaften mit bevorzugter Fahrer-Matching-Priorität.

11.10

> Erstelle eine App-Store-Optimierungs-Checkliste für die Positionierung der App als Pendler-Carpooling-Lösung.

11.11

>Entwirf eine Standort-Berechtigungs-Anfrage für die Ermittlung von Start- und Zielpunkt beim ersten App-Start.

11.12

>Implementiere Hintergrund-Standortverfolgung für die Live-Fahrtverfolgung während der gemeinsamen Fahrt zur Arbeit.

11.13

>Erstelle eine Battery-Optimierungs-Analyse für die dauerhafte GPS-Nutzung während aktiver Mitfahrten.

11.14

>Entwirf eine Widget-Komponente für den Homescreen von der nächsten gebuchten Mitfahrgelegenheit mit Abfahrtszeit und Fahrerinfo.

11.15

>Implementiere Dark Mode für die Fahrten- und Routenübersicht der Pendler-App.

11.16

>Erstelle eine Crash-Reporting-Strategie für die Kartenintegration und den Buchungsprozess der App.

11.17

>Entwirf eine App-Update-Erzwingungs-Logik für kritische Sicherheitsupdates bei der Zahlungsabwicklung zwischen Fahrer und Mitfahrer.

11.18

>Implementiere Kartenintegration für die Anzeige optimaler Sammelpunkte entlang der Pendlerroute.

11.19

>Erstelle eine A/B-Test-Strategie für das Onboarding von neuen Fahrern versus neuen Mitfahrern in der Carpooling-App.

11.20

>Entwirf eine Barrierefreiheitsstrategie für die Buchungs- und Zahlungsabläufe für sehbehinderte Pendler.

12.01

> Erstelle eine Datenpipeline für die Erfassung von Nutzerinteraktionen wie Klicks, Wiedergabedauer und Bewertungen bei einem Streaming-Dienst.

12.02

> Entwirf ein Feature-Engineering-Konzept für Nutzer- und Content-Merkmale wie Genre-Präferenzen, Sehverlauf und Tageszeit der Nutzung.

12.03

> Implementiere ein Empfehlungsmodell für personalisierte Film- und Serienvorschläge auf der Startseite.

12.04

> Erstelle ein A/B-Test-Framework zur Bewertung von zwei konkurrierenden Ranking-Algorithmen für die Reihe „Weil du das gesehen hast“.

12.05

> Definiere Metriken zur Erfolgsmessung von der Empfehlungsqualität, etwa Klickrate, Abschlussquote und Verweildauer.

12.06

> Erstelle ein Datenqualitäts-Monitoring für die eingehenden Rohdaten aus Wiedergabe-Logs und Nutzerbewertungen.

12.07

> Implementiere eine ETL-Strecke für die tägliche Konsolidierung von Sehverlaufsdaten aus mehreren Endgeräten.

12.08

> Entwirf ein Dashboard für Business-Kennzahlen von Abo-Kündigungsrate, durchschnittlicher Sehdauer und Empfehlungs-Klickrate.

12.09

> Erstelle einen Prompt für die Klassifikation von Filmbeschreibungen nach Genre, Stimmung und Zielgruppe.

12.10

> Implementiere eine Anomalieerkennung für ungewöhnliche Spitzen bei Abbruchraten neu veröffentlichter Serien.

12.11

>Erstelle eine Datenanreicherungs-Strategie für Filmmetadaten mit Angaben zu Cast, Regie und Altersfreigabe aus externen Quellen.

12.12

>Entwirf eine Data-Governance-Richtlinie für die Nutzung von Sehverlaufsdaten minderjähriger Nutzerkonten.

12.13

>Implementiere eine Retraining-Pipeline für das Modell von dem wöchentlichen Empfehlungsranking auf Basis neuer Sehgewohnheiten.

12.14

>Erstelle eine Erklärbarkeits-Analyse (Explainability) für die Begründung einzelner Filmempfehlungen gegenüber Endnutzern.

12.15

>Entwirf ein Experiment-Tracking-System für verschiedene Varianten des Empfehlungsalgorithmus im laufenden Testbetrieb.

12.16

>Implementiere eine Datenanonymisierung für das Training von Empfehlungsmodellen auf Basis von Nutzerprofilen.

12.17

>Erstelle ein Modell-Monitoring für Drift bei sich verändernden Sehgewohnheiten nach Veröffentlichung neuer Staffeln.

12.18

>Entwirf eine Kaltstartlösung für neue Nutzer in einem Empfehlungssystem ohne vorhandenen Sehverlauf.

12.19

>Implementiere ein Feedback-Loop-System für explizite Nutzerbewertungen von vorgeschlagenen Filmen und Serien.

12.20

>Erstelle eine Kostenanalyse für die Modellinferenz von Echtzeit-Empfehlungen für Millionen gleichzeitiger Streaming-Nutzer.

13.01

> Erstelle eine README-Struktur für das interne Entwickler-Wiki mit Bereichen für Architektur, Onboarding und Betriebsabläufe.

13.02

> Schreibe eine Schritt-für-Schritt-Anleitung für das Anlegen und Verlinken neuer Seiten im Confluence-basierten Entwickler-Wiki.

13.03

> Erstelle ein Architekturdiagramm mit Erklärung für die Such- und Verlinkungsstruktur des Entwickler-Wikis samt Anbindung an die Git-Repositories des Teams.

13.04

> Dokumentiere die Entscheidung hinter der Wahl von Confluence anstelle eines selbst gehosteten MkDocs-Wikis für die interne Team-Dokumentation.

13.05

> Erstelle ein FAQ-Dokument für die häufigsten Fragen neuer Entwickler:innen zur Nutzung und Struktur des internen Wikis.

13.06

> Schreibe Release Notes für die neu eingeführte Versionsverwaltung von Wiki-Seiten im Entwickler-Wiki.

13.07

> Erstelle ein Troubleshooting-Handbuch für fehlgeschlagene automatische Synchronisationen zwischen dem Wiki und dem zentralen Code-Repository.

13.08

> Dokumentiere die Onboarding-Schritte für neue Mitarbeitende im Bereich Pflege und Erstellung von Inhalten im internen Entwickler-Wiki.

13.09

> Erstelle ein Glossar für Fachbegriffe rund um die Seitenhierarchie, Labels und Berechtigungsstufen des Entwickler-Wikis.

13.10

> Schreibe eine Migrationsanleitung für den Umzug bestehender Team-Dokumentation aus Google Docs in das neue Entwickler-Wiki.

13.11

>Erstelle eine Vorlage für Meeting-Notizen zu quartalsweisen Reviews der Wiki-Inhalte und Dokumentationsstandards.

13.12

>Dokumentiere die Rollen und Verantwortlichkeiten im Team für die Pflege, Qualitätssicherung und Freigabe von Wiki-Artikeln.

13.13

>Erstelle ein Runbook für den Ernstfall bei einem Ausfall des Wiki-Hosting-Dienstes während der Kernarbeitszeit.

13.14

>Schreibe eine Zusammenfassung eines technischen Konzepts für nicht-technische Stakeholder zu der automatisierten Verschlagwortung und Volltextsuche im Entwickler-Wiki.

13.15

>Erstelle eine Checkliste für den Review-Prozess von neu eingereichten Wiki-Artikeln vor deren Veröffentlichung.

13.16

>Dokumentiere bekannte Einschränkungen von der Volltextsuche und der Versionsverwaltung im internen Entwickler-Wiki.

13.17

>Erstelle eine Vorlage für Post-Mortems nach einem Vorfall bei dem versehentlichen Löschen kritischer Architekturseiten durch ein fehlerhaftes Migrationsskript.

13.18

>Schreibe eine Einführung für externe Partner in den eingeschränkten Gastzugang zum internen Entwickler-Wiki.

13.19

>Erstelle eine Versionshistorie-Übersicht für die zentralen Architektur- und Onboarding-Seiten des Entwickler-Wikis.

13.20

>Dokumentiere die Teststrategie für die automatisierten Link- und Konsistenzprüfungen des internen Entwickler-Wikis.

14.01

>Erstelle ein Skript zur automatischen Bereinigung von verwaisten Warenkorb-Sessions in der Checkout-Datenbank.

14.02

>Automatisiere den wiederkehrenden Prozess von monatlichen Abgleichen der Rechnungsdaten zwischen Zahlungsdienstleister und Buchhaltungssystem.

14.03

>Erstelle einen Cronjob für den nächtlichen Export offener Bestellungen aus dem ERP-System.

14.04

>Entwirf einen Slack-Bot für Benachrichtigungen bei Retouren mit ungewöhnlich hohem Warenwert.

14.05

>Erstelle ein Skript zur Massенbearbeitung von Produktpreisen während saisonaler Rabattaktionen.

14.06

>Automatisiere die Erstellung von Reports für die wöchentliche Auswertung von Versandverzögerungen nach Spediteur.

14.07

>Erstelle eine CLI für das schnelle Nachschlagen von Lagerbeständen und Lieferantendaten.

14.08

>Entwirf einen Workflow in n8n/Zapier für die automatische Weiterleitung von Kundenreklamationen aus dem Support-Postfach an das zuständige Team.

14.09

>Erstelle ein Skript zur Validierung von Importdateien für neue Produktkataloge von Lieferanten vor dem Upload in den Onlineshop.

14.10

>Automatisiere das Zurücksetzen der Testumgebung für das Checkout- und Zahlungsabwicklungssystem.

14.11

>Erstelle einen Watcher, der bei Änderungen an den Lagerbestandsdaten im Warenwirtschaftssystem reagiert.

14.12

>Entwirf eine Automatisierung für das Onboarding neuer Kunden in dem B2B-Großhandelsportal.

14.13

>Erstelle ein Skript zur Erkennung verwaister Datensätze in der Kundendatenbank nach gelöschten Bestellungen.

14.14

>Automatisiere die Synchronisation zwischen dem ERP-System und dem Amazon-Seller-Konto.

14.15

>Erstelle ein Skript für die automatische Kategorisierung von eingehenden Kundenanfragen im Support-Postfach.

14.16

>Entwirf eine Eskalations-Automatisierung für Bestellungen, die die zugesagte Lieferzeit überschreiten.

14.17

>Erstelle ein Skript zur automatischen Erstellung von Testdaten für Lasttests des Checkout-Prozesses.

14.18

>Automatisiere die Versandbenachrichtigung bei dem Statuswechsel einer Bestellung auf „versandt“.

14.19

>Erstelle eine Automatisierung für wiederkehrende Rechnungen in Abonnement-Bestellungen im Onlineshop.

14.20

>Entwirf eine selbstheilende Automatisierung (Self-Healing) für fehlgeschlagene Bestellübertragungen an Marktplätze wie Amazon und eBay.

15.01

> Erstelle eine Cloud-Architektur für eine mandantenfähige SaaS-Plattform mit Microservices, die aus einem On-Premise-Rechenzentrum in eine Public-Cloud-Umgebung migriert wird.

15.02

> Definiere eine Multi-Cloud-Strategie für die Verteilung der Workloads der SaaS-Plattform auf AWS und Azure zur Vermeidung eines einzelnen Ausfallpunkts.

15.03

> Erstelle ein Terraform-Modul für die Bereitstellung der Kubernetes-Cluster und Datenbank-Instanzen der migrierten SaaS-Plattform.

15.04

> Entwirf eine Serverless-Architektur für die Verarbeitung von Nutzer-Uploads und asynchronen Hintergrundjobs der SaaS-Plattform.

15.05

> Erstelle eine Kostenanalyse für die Cloud-Ressourcen von der wachsenden SaaS-Plattform im ersten Jahr nach der Migration aus dem eigenen Rechenzentrum.

15.06

> Definiere eine Auto-Scaling-Richtlinie für die Anwendungsserver der SaaS-Plattform bei schwankender Nutzerlast während der Stoßzeiten.

15.07

> Erstelle eine Netzwerkarchitektur (VPC) für die Trennung von Produktions-, Staging- und Entwicklungsumgebungen der migrierten SaaS-Plattform.

15.08

> Entwirf eine Disaster-Recovery-Strategie über mehrere Regionen für die Kundendatenbanken der SaaS-Plattform nach dem Wegfall des bisherigen On-Premise-Backup-Rechenzentrums.

15.09

>Erstelle eine Reserved-Instance-Strategie zur Kostensenkung von den dauerhaft laufenden Compute-Instanzen der SaaS-Plattform in der neuen Cloud-Umgebung.

15.10

>Definiere eine Tagging-Konvention für Cloud-Ressourcen von den einzelnen Teams und Produktmodulen der migrierten SaaS-Plattform zur Kostenzuordnung.

15.11

>Erstelle eine Migrationsstrategie von On-Premise in die Cloud für die bestehende relationale Datenbank und den Anwendungsserver der SaaS-Plattform bei laufendem Betrieb.

15.12

>Entwirf eine Container-Orchestrierungsstrategie für die Microservices der SaaS-Plattform nach der Ablösung der monolithischen On-Premise-Anwendung.

15.13

>Erstelle eine Backup- und Restore-Strategie für die Kundendaten und Konfigurationsstände der SaaS-Plattform nach der Cloud-Migration.

15.14

>Definiere eine Zugriffsrichtlinie (IAM) für die Entwickler- und Betriebsteams, die nach dem Umzug in die Cloud auf die Produktionsumgebung der SaaS-Plattform zugreifen.

15.15

>Erstelle eine Edge-Computing-Strategie für die Auslieferung statischer Inhalte und die Reduzierung der Latenz für internationale Kunden der SaaS-Plattform.

15.16

>Entwirf eine Hybrid-Cloud-Lösung für die Übergangsphase, in der sensible Altsysteme noch On-Premise verbleiben, während neue Dienste der SaaS-Plattform bereits in der Cloud laufen.

15.17

>Erstelle ein Kostenüberwachungs-Dashboard für die laufenden Cloud-Ausgaben der SaaS-Plattform im Vergleich zu den früheren On-Premise-Betriebskosten.

15.18

>Definiere eine Strategie zur Reduzierung von Vendor-Lock-in für die Datenbank- und Speicherdienste der SaaS-Plattform nach der Migration zu einem einzelnen Cloud-Anbieter.

15.19

>Erstelle eine Compliance-Checkliste für Cloud-Ressourcen von der SaaS-Plattform zur Einhaltung der DSGVO nach dem Umzug der Kundendaten in die Cloud.

15.20

>Entwirf eine nachhaltige (grüne) Infrastrukturstrategie für die Rechenzentren der Cloud-Anbieter, die künftig die Workloads der SaaS-Plattform betreiben.

\$ Abschluss

Diese 300 Masterprompts sind als Ausgangspunkt gedacht, nicht als starres Regelwerk. Passe Formulierungen, Umfang und Detailgrad an dein Projekt, dein Team und deinen Tech-Stack an — je präziser der Kontext, desto treffsicherer das Ergebnis. Fortgeschrittene Nutzer kombinieren mehrere Prompts zu Ketten (z. B. Architektur → Code → Tests → Deployment), lassen Claude Code Zwischenergebnisse kritisch prüfen und bauen sich eigene Bibliotheken projektspezifischer Varianten auf. Je öfter du die Prompts wiederverwendest und anpasst, desto mehr entwickeln sie sich zu einem persönlichen Werkzeugkasten für effizientere Softwareentwicklung.

\$ 300 Masterprompts · 15 Kategorien · erstellt für Claude Code